

Software Detailed Design (SDD) for Rental Property Management System — Version 1.0

By GROUP 12

Table of Contents

I. Introduction	5
I.1. Purpose	5
I.2. Scope	5
I.3. Intended Audience	5
I.4. Definitions and Abbreviations	6
II. References	6
III. System Architecture Overview	8
III.1. Architectural Style	8
III.2. Component Map	9
III.3. Cross-Cutting Concerns	10
IV. Database Design	11
IV.1. Entity-Relationship Overview	11
IV.2. Table: users	11
IV.3. Table: rooms	13
IV.4. Table: lease_contracts	14
IV.5. Table: meter_readings	16
IV.6. Table: invoices	18
IV.7. Table: maintenance_requests	20
IV.8. Table: notifications	21
IV.9. Table: refresh_tokens	22
IV.10. Table: revoked_access_tokens	24
IV.11. Table: audit_logs	24
V. Inter-Component Workflow Descriptions	26
V.1. Tenant Management Component (C-01)	26
V.1.1. Purpose	26
V.1.2. Interfaces	26
V.1.3. Processing Logic	28
V.1.4. Database Interactions	28
V.2. Room Management Component (C-02)	29

V.2.1. Purpose	29
V.2.2. Interfaces	29
V.2.3. Room State Machine.....	30
V.2.4. Cascade Delete Algorithm.....	31
V.2.5. Database Interactions	31
V.3. Contract Management Component (C-03).....	32
V.3.1. Purpose	32
V.3.2. Interfaces	32
V.3.3. Contract Creation Algorithm	34
V.3.4. Contract Termination Algorithm	35
V.3.5. Contract Lifecycle State Machine.....	35
V.4. Billing & Invoicing Component (C-04)	36
V.4.1. Purpose	36
V.4.2. Interfaces	36
V.4.3. Invoice Calculation Algorithm	37
V.4.4. Previous Meter Reading Retrieval Algorithm	39
V.4.5. Invoice Creation Workflow	40
V.4.6. Payment Recording Algorithm.....	41
V.4.7. Invoice Deletion Algorithm.....	42
V.5. Tenant Portal Component (C-05)	42
V.5.1. Purpose	42
V.5.2. Interfaces	42
V.5.3. Tenant Invoice Query Algorithm	43
V.5.4. Maintenance Submission Algorithm	44
V.6. Reporting & Analytics Component (C-06)	45
V.6.1. Purpose	45
V.6.2. Interfaces	45
V.6.3. Financial Statistics Algorithm	46
V.6.4. Temporary Residence Report Generation Algorithm	47
V.6.5. Tax Report Generation Algorithm	48
V.7. Authentication Service Component (C-07).....	48

V.7.1. Purpose	48
V.7.2. Token Architecture	49
V.7.3. Token Validation Middleware	50
V.7.4. Login Algorithm	50
V.7.5. Refresh Token Rotation Algorithm	51
V.7.6. Logout Algorithm	52
V.7.7. Token Cleanup Background Job	52
V.8. Notification Service Component (C-08)	52
V.8.1. Purpose	52
V.8.2. Interfaces	52
V.8.3. Notification Dispatch Algorithm	53
V.8.4. Notification Retrieval Logic	54
V.9. Audit & Logging Component (C-09)	54
V.9.1. Purpose	54
V.9.2. Interfaces	55
V.9.3. Middleware Audit Capture	55
V.9.4. Audit Log Query Algorithm	56
VI. Inter-Component Workflow Descriptions	57
VI.1. Monthly Billing Workflow (UC-02)	57
VI.2. Tenant Registration & Approval Workflow (UC-09, UC-01)	57
VI.3. Contract Creation & Room Status Transition Workflow (UC-08)	58
VI.4. QR Code Payment Workflow (UC-03)	58
VI.5. Maintenance Request Priority Escalation Workflow (UC-04)	59
VII. COTS/SOUP Component Identification	59
VII.1. Backend Dependencies	59
VII.2. Backend Dependencies	60
VII.3. Vietnamese Font Files (Static Assets)	61
VIII. Critical Requirements Traceability	61
IX. Design Constraints and Assumptions	64

I. Introduction

I.1. Purpose

This Software Detailed Design (SDD) document describes the internal architecture, component structure, data schemas, interface specifications, algorithmic logic, and inter-component workflows of the Rental Property Management System (RPMS). It translates the functional requirements captured in the SRS v1.0 and the Business Rules of Document 3 into a concrete technical blueprint sufficient for implementation, code review, and system testing.

I.2. Scope

This document covers the complete RPMS application stack, comprising:

- A **Node.js/Express.js RESTful backend API** served on port 5000
- A **SQLite relational database** (rental.db) persisted at a configurable path
- A **React/Vite single-page frontend application** served from a Vite development server (port 5173 in development)
- Nine discrete software components spanning authentication, room management, tenant management, contract management, billing and invoicing, the tenant portal, reporting and analytics, the notification service, and audit logging

The system is designed for deployment by individual boarding house owners managing single or multiple properties in Vietnam. All user interface text and system messages are in Vietnamese.

I.3. Intended Audience

Audience	Usage
Backend Developers	API implementation, database migrations, service logic
Frontend Developers	Component integration, API consumption, state management

QA Engineers	Test case design, boundary condition identification
System Architects	Review of component boundaries, dependency management
Technical Reviewers	Inspection of security, data integrity, and compliance logic

I.4. Definitions and Abbreviations

Term	Definition
RPMS	Rental Property Management System
JWT	JSON Web Token — stateless authentication credential
JTI	JWT ID — unique identifier embedded in each token
OTP	One-Time Password — single-use code for identity verification
CRUD	Create, Read, Update, Delete — standard data operations
SRS	Software Requirements Specification
BR	Business Rule (Document 3)
UC	Use Case (referenced from SRS 3)
HTTP	HyperText Transfer Protocol
PDF	Portable Document Format
XLSX	Microsoft Excel Open XML Spreadsheet format
VNĐ	Vietnamese Dong — currency unit
kWh	Kilowatt-hour — electricity consumption unit
m ³	Cubic metre — water consumption unit
CORS	Cross-Origin Resource Sharing
COTS	Commercial Off-The-Shelf software
SOUP	Software of Unknown Provenance — third-party libraries

II. References

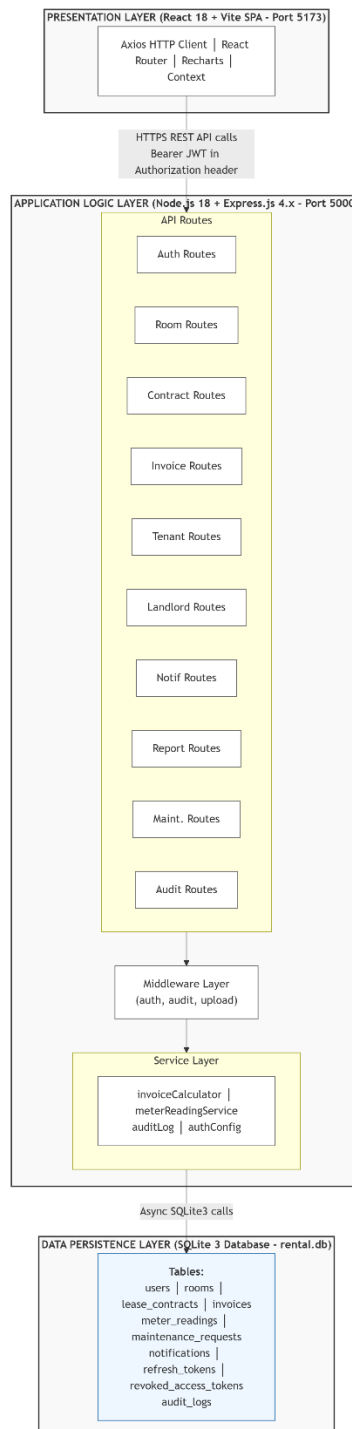
1. Team 12. *Software requirements specification for RPMS* (Version 1.0).
2. Team 12. *Business rules document for RPMS* (Version 1.0).
3. Team 12. *Sorted requirements worksheet mapping* (Version 1.0).
4. Express. (n.d.). *Express.js documentation* (Version 4.x). <https://expressjs.com>

5. TryGhost. (n.d.). *SQLite3 Node.js documentation* (Version 5.x). GitHub. <https://github.com/TryGhost/node-sqlite3>
6. Internet Engineering Task Force. (2015). *JSON Web Token (JWT) specification* (RFC 7519). Internet Datatracker. <https://datatracker.ietf.org/doc/html/rfc7519>
7. dcodeIO. (n.d.). *bcryptjs documentation* (Version 2.x). GitHub. <https://github.com/dcodeIO/bcrypt.js>
8. Natancabral. (n.d.). *PDFKit table documentation*. GitHub. <https://github.com/natancabral/pdfkit-table>
9. ExcelJS. (n.d.). *ExcelJS documentation* (Version 4.x). GitHub. <https://github.com/exceljs/exceljs>
10. Meta Open Source. (n.d.). *React documentation* (Version 18.x). <https://react.dev>
11. Recharts. (n.d.). *Recharts documentation* (Version 2.x). <https://recharts.org>

III. System Architecture Overview

III.1. Architectural Style

The RPMS employs a **three-tier client-server architecture** with a clear separation between the presentation layer (React SPA), the application logic layer (Express.js API), and the data persistence layer (SQLite database).



III.2. Component Map

Component ID	Component Name	Primary Responsibility	Source Files
C-01	Tenant Management	Account lifecycle, profile CRUD, approval workflow	routes/tenants.js, routes/landlord.js
C-02	Room Management	Room CRUD, status tracking, occupancy statistics	routes/rooms.js
C-03	Contract Management	Lease contract lifecycle, room transfer, alerts	routes/contracts.js
C-04	Billing & Invoicing	Meter readings, invoice computation, payment recording	routes/invoices.js, routes/meter-readings.js, routes/landlord.js, services/invoiceCalculator.js, services/meterReadingService.js
C-05	Tenant Portal	Tenant-facing views: contract, invoices, maintenance, profile	routes/tenants.js (tenant endpoints)
C-06	Reporting & Analytics	Financial dashboards, PDF/XLSX legal and tax reports	routes/reports.js, routes/landlord.js (financial-stats)

C-07	Authentication Service	JWT issuance, validation, rotation, revocation, rate limiting	routes/auth.js, middleware/auth.js, config/auth.js
C-08	Notification Service	Automated alerts, broadcast messaging, read/delete	routes/notifications.js
C-09	Audit & Logging	Mutation capture, log query, retention	routes/audit.js, middleware/audit.js, services/auditLog.js

III.3. Cross-Cutting Concerns

Concern	Implementation
Authentication	authenticateToken middleware applied to all protected routes
Authorisation	requireRole(role) middleware applied per route group
Rate Limiting	express-rate-limit applied globally at server entry point
CORS	Configurable allowed-origins list via FRONTEND_URL environment variable
Error Handling	Global app.use((err, req, res, next) => {...}) catch-all handler
File Uploads	multer middleware with disk and memory storage strategies
Database	Promisified SQLite3 wrapper (db.runAsync, db.getAsync, db.allAsync) with PRAGMA foreign_keys = ON
Graceful Shutdown	SIGTERM/SIGINT handlers close HTTP server and SQLite connection cleanly

IV. Database Design

IV.1. Entity-Relationship Overview

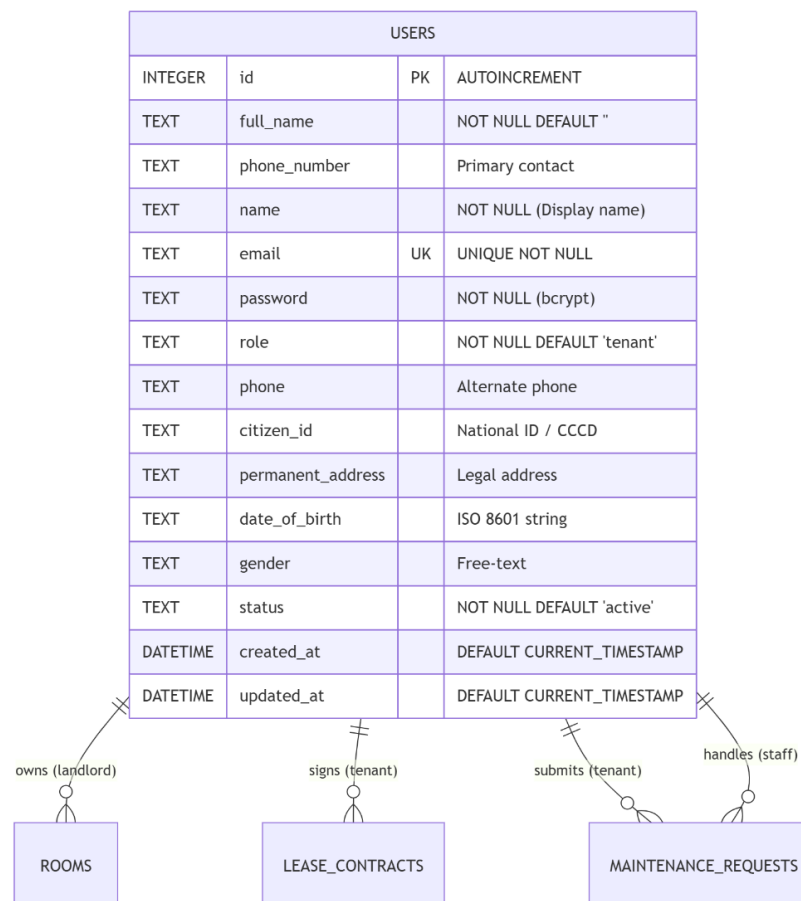
The database contains eleven primary tables. The following describes each entity, its attributes, data types, constraints, and relationships.

IV.2. Table: users

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
full_name	TEXT	NOT NULL DEFAULT " (null characters)	Legal full name
phone_number	TEXT	—	Primary contact number
name	TEXT	NOT NULL	Display name (alias for full_name)
email	TEXT	UNIQUE NOT NULL	Login credential; immutable after creation
password	TEXT	NOT NULL	bcrypt-hashed password (10 salt rounds)
role	TEXT	NOT NULL DEFAULT 'tenant'	One of: landlord, tenant, Owner, Manager, Tenant, TechnicalStaff
phone	TEXT	—	Alternate phone field
citizen_id	TEXT	—	National ID / CCCD number (max 20 chars for tenants)

permanent_address	TEXT	—	Legal registered address
date_of_birth	TEXT	—	ISO 8601 date string
gender	TEXT	—	Free-text gender designation
status	TEXT	NOT NULL DEFAULT 'active'	One of: pending, active, inactive
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Last modification timestamp

ERD:

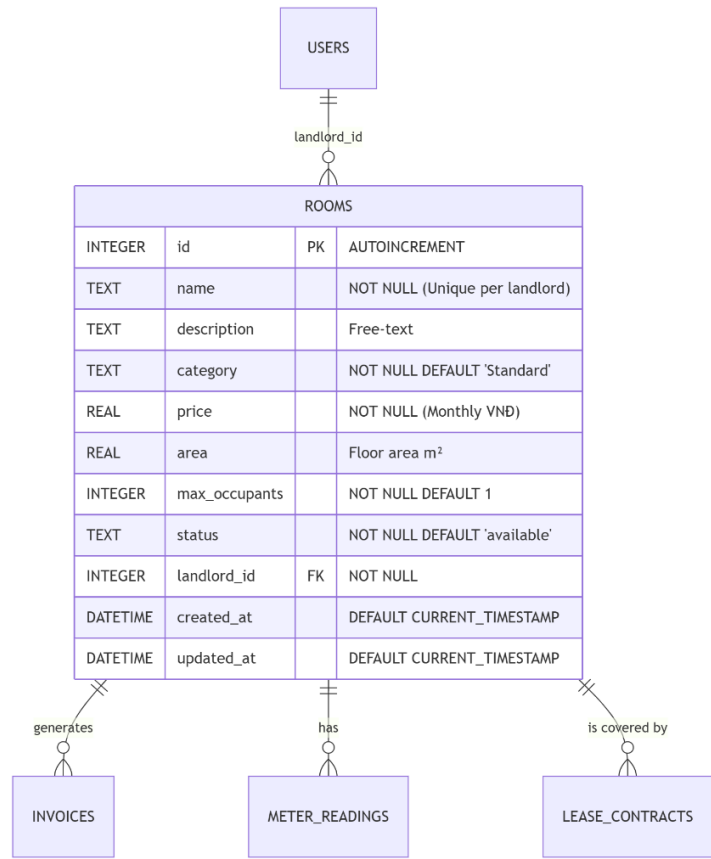


IV.3. Table: rooms

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
name	TEXT	NOT NULL	Room identifier (unique per landlord)
description	TEXT	—	Free-text room description
category	TEXT	NOT NULL DEFAULT 'Standard'	Classification: Standard, Loft, Fully Furnished, Demo
price	REAL	NOT NULL	Monthly rental price in VNĐ (≥ 0)
area	REAL	—	Floor area in m ²
max_occupants	INTEGER	NOT NULL DEFAULT 1	Maximum permitted occupants (≥ 1)
status	TEXT	NOT NULL DEFAULT 'available'	One of: available, occupied, maintenance, reserved, cleaning
landlord_id	INTEGER	NOT NULL, FK → users(id)	Owning landlord
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—
------------	----------	------------------------------	---

ERD:

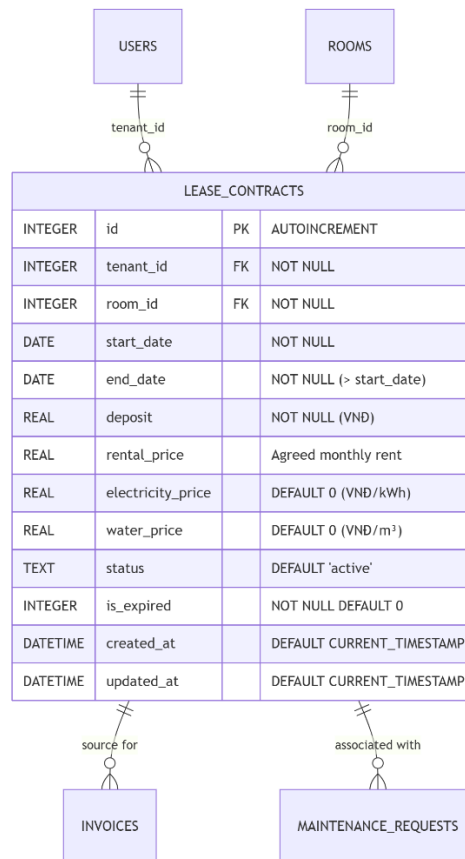


IV.4. Table: lease contracts

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
tenant_id	INTEGER	NOT NULL, FK → users(id)	Contracting tenant
room_id	INTEGER	NOT NULL, FK → rooms(id)	Contracted room
start_date	DATE	NOT NULL	Contract commencement date (ISO 8601)

end_date	DATE	NOT NULL	Contract expiry date (ISO 8601); must be > start_date
deposit	REAL	NOT NULL	Security deposit in VNĐ (≥ 0)
rental_price	REAL	—	Agreed monthly rent in VNĐ (> 0)
electricity_price	REAL	DEFAULT 0	Per-unit electricity price (VNĐ/kWh)
water_price	REAL	DEFAULT 0	Per-unit water price (VNĐ/m ³)
status	TEXT	DEFAULT 'active'	One of: active, terminated, expired
is_expired	INTEGER	NOT NULL DEFAULT 0	Boolean flag: 1 = expired
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

ERD:



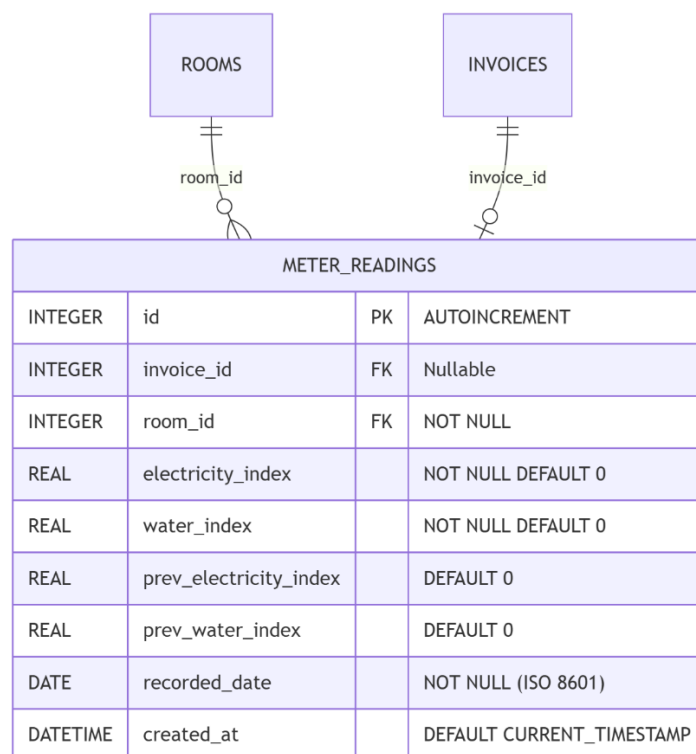
Indexes: idx_lease_contracts_tenant_id, idx_lease_contracts_room_id

IV.5. Table: meter_readings

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
invoice_id	INTEGER	FK → invoices(id)	Linked invoice (nullable before invoice creation)
room_id	INTEGER	NOT NULL, FK → rooms(id)	Room being measured
electricity_index	REAL	NOT NULL DEFAULT 0	Current period electricity meter value (kWh)

water_index	REAL	NOT NULL DEFAULT 0	Current period water meter value (m³)
prev_electricity_index	REAL	DEFAULT 0	Previous period electricity value (audit trail)
prev_water_index	REAL	DEFAULT 0	Previous period water value (audit trail)
recorded_date	DATE	NOT NULL	Date the reading was taken (ISO 8601)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

ERD:



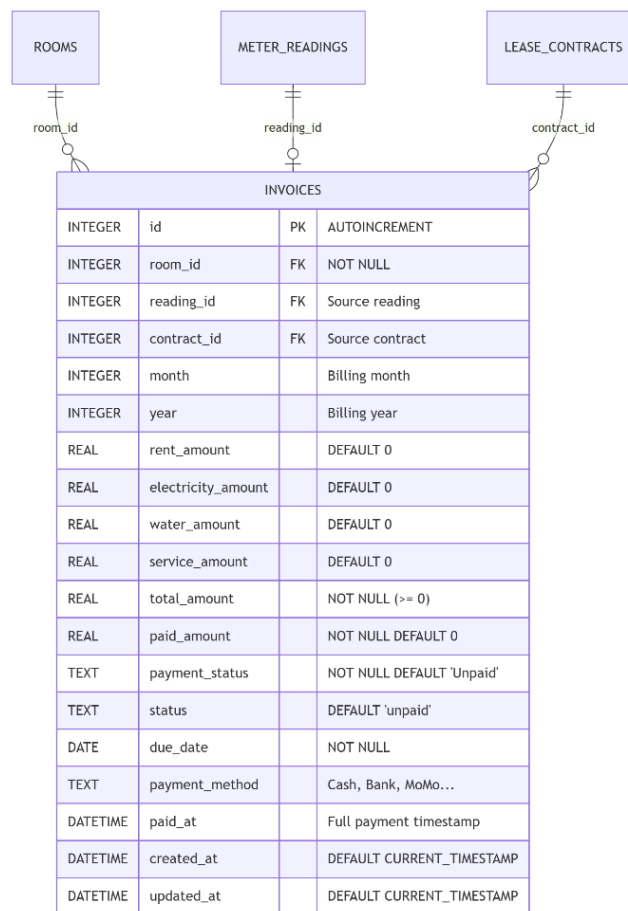
Indexes: idx_meter_readings_room_id

IV.6. Table: invoices

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
room_id	INTEGER	NOT NULL, FK → rooms(id)	Invoiced room
reading_id	INTEGER	FK → meter_readings(id)	Source meter reading
contract_id	INTEGER	FK → lease_contracts(id)	Source contract
month	INTEGER	—	Billing month (1–12)
year	INTEGER	—	Billing year (e.g., 2026)
rent_amount	REAL	DEFAULT 0	Room rental charge
electricity_amount	REAL	DEFAULT 0	Computed electricity charge
water_amount	REAL	DEFAULT 0	Computed water charge
service_amount	REAL	DEFAULT 0	WiFi + garbage + other service fees
total_amount	REAL	NOT NULL	Grand total (≥ 0)
paid_amount	REAL	NOT NULL DEFAULT 0	Cumulative amount received
payment_status	TEXT	NOT NULL DEFAULT 'Unpaid'	Display status: Unpaid, Paid, Partial
status	TEXT	DEFAULT 'unpaid'	System status: unpaid, partial, paid
due_date	DATE	NOT NULL	Payment deadline

payment_method	TEXT	—	Cash, Bank Transfer, MoMo, ZaloPay, Mock QR Transfer
paid_at	DATETIME	—	Timestamp of full payment completion
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

ERD:



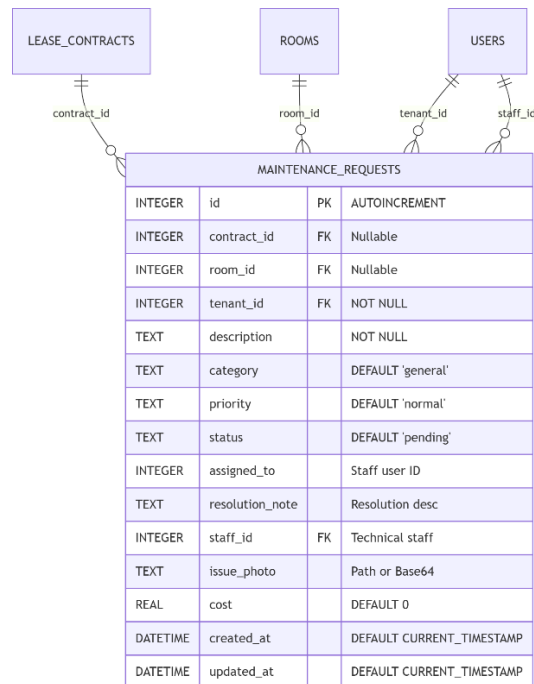
Indexes: idx_invoices_room_id, idx_invoices_reading_id

IV.7. Table: maintenance_requests

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
contract_id	INTEGER	FK → lease_contracts(id)	Associated contract (nullable)
room_id	INTEGER	FK → rooms(id)	Room requiring maintenance
tenant_id	INTEGER	NOT NULL, FK → users(id)	Requesting tenant
description	TEXT	NOT NULL	Issue description (non- empty)
category	TEXT	DEFAULT 'general'	One of: electrical, plumbing, air_conditioning, furniture, internet, general
priority	TEXT	DEFAULT 'normal'	One of: normal, high
status	TEXT	DEFAULT 'pending'	One of: pending, in_progress, completed, cancelled
assigned_to	INTEGER	—	Assigned staff user ID
resolution_note	TEXT	—	Resolution description
staff_id	INTEGER	FK → users(id)	Technical staff assigned
issue_photo	TEXT	—	File path or Base64- encoded image
cost	REAL	DEFAULT 0	Cost of repair in VNĐ
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP AMP	—
------------	----------	-------------------------------------	---

ERD:



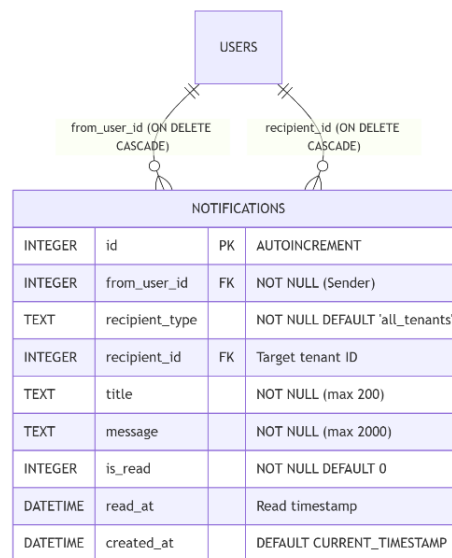
Indexes: idx_maintenance_requests_tenant_id,
idx_maintenance_requests_staff_id

IV.8. Table: notifications

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate primary key
from_user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Sender (landlord/manager)
recipient_type	TEXT	NOT NULL DEFAULT 'all_tenants'	One of: all_tenants, room, tenant, selected_tenants

recipient_id	INTEGER	FK → users(id) ON DELETE CASCADE	Target tenant ID (when type is tenant)
title	TEXT	NOT NULL	Notification subject (max 200 chars)
message	TEXT	NOT NULL	Notification body (max 2000 chars)
is_read	INTEGER	NOT NULL DEFAULT 0	Boolean: 0 = unread, 1 = read
read_at	DATETIME	—	Timestamp when marked as read
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

ERD:



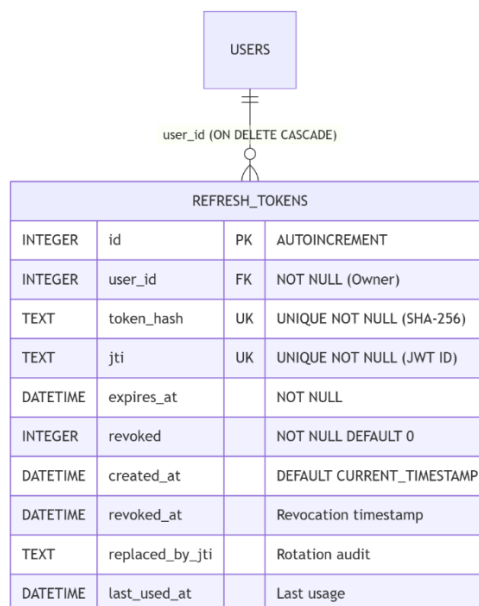
Indexes: idx_notifications_from_user_id, idx_notifications_recipient_id, idx_notifications_recipient_type, idx_notifications_is_read

IV.9. Table: refresh_tokens

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	—

user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Token owner
token_hash	TEXT	UNIQUE NOT NULL	SHA-256 hash of the raw refresh token
jti	TEXT	UNIQUE NOT NULL	JWT ID of this refresh token
expires_at	DATETIME	NOT NULL	Absolute expiry timestamp
revoked	INTEGER	NOT NULL DEFAULT 0	Boolean: 1 = revoked
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—
revoked_at	DATETIME	—	Revocation timestamp
replaced_by_jti	TEXT	—	JTI of the replacement token (rotation audit)
last_used_at	DATETIME	—	Last usage timestamp

ERD:

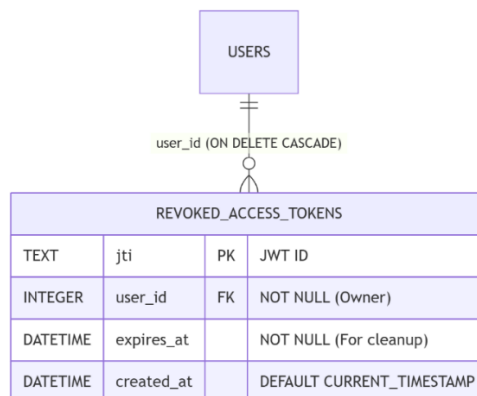


Indexes: idx_refresh_tokens_user_id, idx_refresh_tokens_token_hash

IV.10. Table: revoked access tokens

Column	Type	Constraints	Description
jti	TEXT	PRIMARY KEY	JWT ID of the revoked access token
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Token owner
expires_at	DATETIME	NOT NULL	Original token expiry (used for cleanup)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Revocation timestamp

ERD:



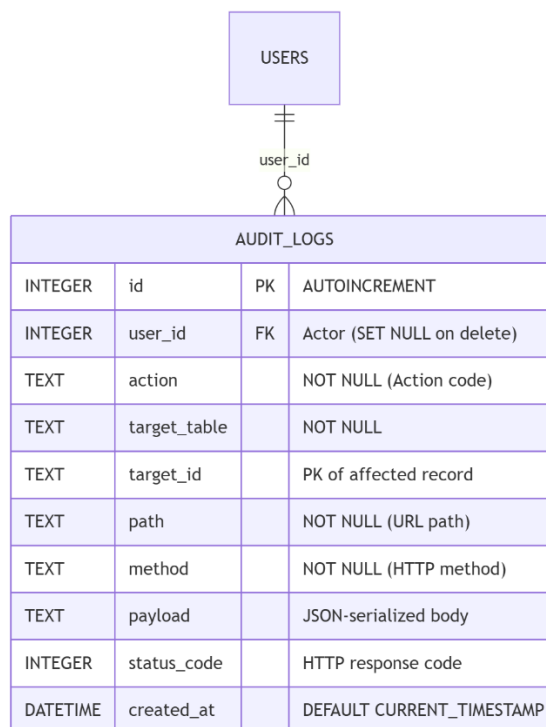
Indexes: idx_revoked_access_expires_at

IV.11. Table: audit logs

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	—
user_id	INTEGER	FK → users(id) ON DELETE SET NULL	Actor who performed the action
action	TEXT	NOT NULL	Action code: e.g., CREATE_INVOICE,

			DELETE_ROOM, PAY_INVOICE
target_table	TEXT	NOT NULL	Affected database table name
target_id	TEXT	—	Primary key of the affected record
path	TEXT	NOT NULL	Request URL path
method	TEXT	NOT NULL	HTTP method: PUT, DELETE
payload	TEXT	—	JSON-serialised request body/params/query
status_code	INTEGER	—	HTTP response code (captured post-response)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

ERD:



Indexes: idx_audit_logs_user_id, idx_audit_logs_created_at

V. Inter-Component Workflow Descriptions

V.1. Tenant Management Component (C-01)

V.1.1. Purpose

Manages the full lifecycle of user accounts, specifically Tenant registration, approval, deactivation, and profile management, as well as the creation and management of Manager accounts by the Owner.

V.1.2. Interfaces

Input Interfaces (API Endpoints consumed by this component):

Method	Endpoint	Auth Required	Role	Description
GET	/api/tenants	Yes	landlord	Retrieve all tenant accounts
GET	/api/tenants/all	Yes	landlord	Retrieve all tenants with active-contract flag
GET	/api/tenants/account-list	Yes	landlord	Retrieve tenants with active contracts for notifications
POST	/api/tenants	Yes	landlord	Create a new tenant account directly
GET	/api/tenants/profile	Yes	tenant/landlord	Retrieve own profile
PUT	/api/tenants/profile	Yes	tenant/landlord	Update own profile
PUT	/api/tenants/:id/status	Yes	landlord	Approve, lock, or unlock a tenant account
GET	/api/tenants/:id	Yes	landlord	Retrieve a specific tenant's details

PUT	/api/tenants/:id	Yes	landlord	Update a specific tenant's details
DELETE	/api/tenants/:id	Yes	landlord	Delete a tenant account
GET	/api/landlord/tenants	Yes	landlord	List tenants with active contracts
GET	/api/landlord/tenants/all	Yes	landlord	Search all tenant accounts
GET	/api/landlord/tenants/:id	Yes	landlord	Tenant detail with contract history

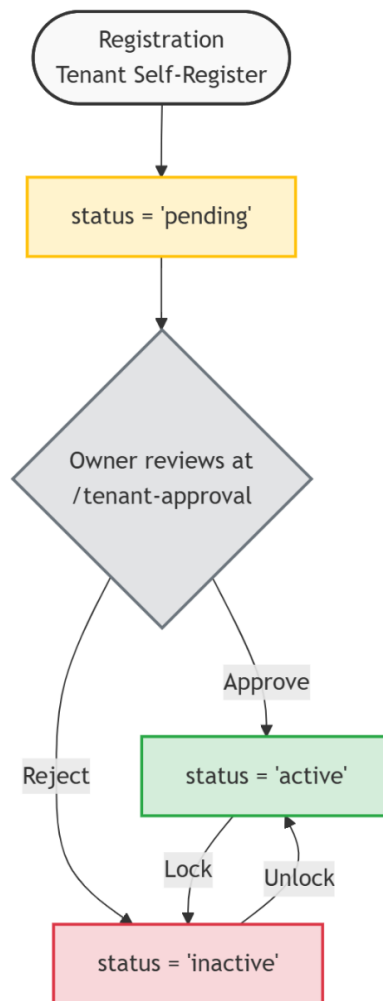
Output Data Structures:

```

TenantRecord {
  id: Integer,
  fullName: String,
  email: String,
  phoneNumber: String,
  citizenID: String | null,
  permanentAddress: String | null,
  dateOfBirth: String | null,
  gender: String | null,
  status: "pending" | "active" | "inactive",
  createdAt: ISO8601DateTime,
  contracts: ContractSummary[] // returned in detail
endpoint
}

```

V.1.3. Processing Logic



- **Direct Creation by Owner:** When the Owner creates a Tenant directly via POST /api/tenants, the account is created with status = 'active' (bypassing the pending-approval flow), enabling immediate contract assignment.
- **Profile Update Constraints:** The Tenant profile endpoint (PUT /api/tenants/profile) is accessible to both tenant and landlord roles, updating the name, phone, citizen_id, permanent_address, date_of_birth, and gender fields on the authenticated user's own record.

V.1.4. Database Interactions

Operation	SQL Target	Trigger
List all tenants	SELECT FROM users WHERE role IN ('tenant','Tenant')	Owner views tenant list

Update account status	UPDATE users SET status=? WHERE id=?	Owner approves/locks account
Retrieve own profile	SELECT FROM users WHERE id = req.user.id	Tenant accesses portal
Update own profile	UPDATE users SET name=?, phone=?, ... WHERE id=?	Tenant/Owner edits profile
List active-contract tenants	JOIN users, lease_contracts, rooms WHERE landlord_id=? AND status='active'	Notification targeting

V.2. Room Management Component (C-02)

V.2.1. Purpose

Provides complete CRUD management for physical rental units, enforces status state-machine transitions, and exposes occupancy statistics.

V.2.2. Interfaces

Input Interfaces:

Method	Endpoint	Auth	Role	Description
GET	/api/rooms	Yes	landlord	List all rooms (filter by status, category)
GET	/api/rooms/available	Yes	landlord	List available rooms only
POST	/api/rooms	Yes	landlord	Create a new room
PUT	/api/rooms/:id	Yes	landlord	Update room attributes
DELETE	/api/rooms/:id	Yes	landlord	Delete room and cascade-delete related records
POST	/api/rooms/seed-demo	Yes	landlord	Seed demonstration rooms

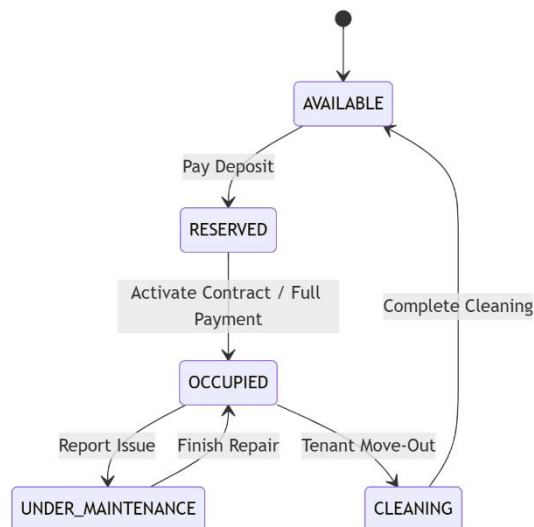
Output Data Structure:

```

RoomRecord {
  roomId: Integer,
  id: Integer,          // alias for roomId
  name: String,
  description: String,
  category: String,
  price: Float,
  area: Float,
  maxOccupants: Integer,
  capacity: Integer,    // alias for maxOccupants
  status: "available" | "occupied" | "maintenance" |
"reserved" | "cleaning",
  landlordName: String,
  createdAt: ISO8601DateTime,
  updatedAt: ISO8601DateTime
}

```

V.2.3. Room State Machine



Automatic Transitions (System-Enforced):

Trigger	From Status	To Status	Implementation
Contract creation succeeds	available	occupied	contracts.js POST /
Contract termination succeeds	occupied	available	contracts.js PUT /:id/terminate

V.2.4. Cascade Delete Algorithm

When DELETE /api/rooms/:id is invoked:

```
BEGIN TRANSACTION
  1. Verify room exists AND room.landlord_id = req.user.id
  2. SELECT all invoices WHERE room_id = roomId
  3. FOR each invoice:
      DELETE FROM meter_readings WHERE invoice_id =
invoice.id
  4. DELETE FROM invoices WHERE room_id = roomId
  5. DELETE FROM meter_readings WHERE room_id = roomId
(any unlinked readings)
  6. DELETE FROM rooms WHERE id = roomId AND landlord_id =
req.user.id
COMMIT
```

If any step fails, ROLLBACK is executed and HTTP 500 is returned.

V.2.5. Database Interactions

Operation	SQL Statement Pattern	Notes
Create room	INSERT INTO rooms (name, description, ...) VALUES (...)	Checks name uniqueness first
Name uniqueness check	SELECT id FROM room WHERE name=? AND landlord_id=?	Returns 400 on conflict
Update room	UPDATE rooms SET name=?, price=?, status=?, ... WHERE id=? AND landlord_id=?	Validates status is in VALID_ROOM_STATUSES
List by filter	SELECT ... FROM rooms WHERE landlord_id=? [AND status=?] [AND category=?]	Dynamic WHERE clause
Cascade delete	Transaction across rooms, invoices, meter_readings	See algorithm above

V.3. Contract Management Component (C-03)

V.3.1. Purpose

Manages the full lifecycle of lease agreements from creation to termination, enforces the one-active-contract-per-tenant rule, and handles the room transfer workflow.

V.3.2. Interfaces

Method	Endpoint	Auth	Role	Description
POST	/api/contracts	Yes	landlord	Create new lease contract
GET	/api/contracts	Yes	landlord	List all contracts (filter by status)
GET	/api/contracts/my-contract	Yes	any	Retrieve own active contract (tenant view)
GET	/api/contracts/:id	Yes	landlord/tenant	Contract detail
PUT	/api/contracts/:id/terminate	Yes	landlord	Terminate an active contract

Input Schema for Contract Creation:

```
CreateContractRequest {  
  roomId: Integer (required),  
  tenantID: Integer (required),  
  startDate: ISO8601Date (required),  
  endDate: ISO8601Date (required, > startDate),  
  deposit: Float (required, ≥ 0),  
  rentalPrice: Float (required, > 0),  
  electricity_price: Float (optional, default 0),  
  water_price: Float (optional, default 0)  
}
```

Output Data Structure:


```
ContractRecord {  
  contractID: Integer,  
  roomID: Integer,  
  tenantID: Integer,  
  startDate: ISO8601Date,  
  endDate: ISO8601Date,  
  deposit: Float,  
  rentalPrice: Float,  
  electricityPrice: Float,  
  waterPrice: Float,  
  status: "active" | "terminated" | "expired",  
  isExpired: Boolean,  
  createdAt: ISO8601DateTime,  
  roomName: String,  
  roomPrice: Float,  
  tenantName: String,  
  tenantEmail: String,  
  tenantPhone: String,  
  tenantCitizenID: String  
}
```

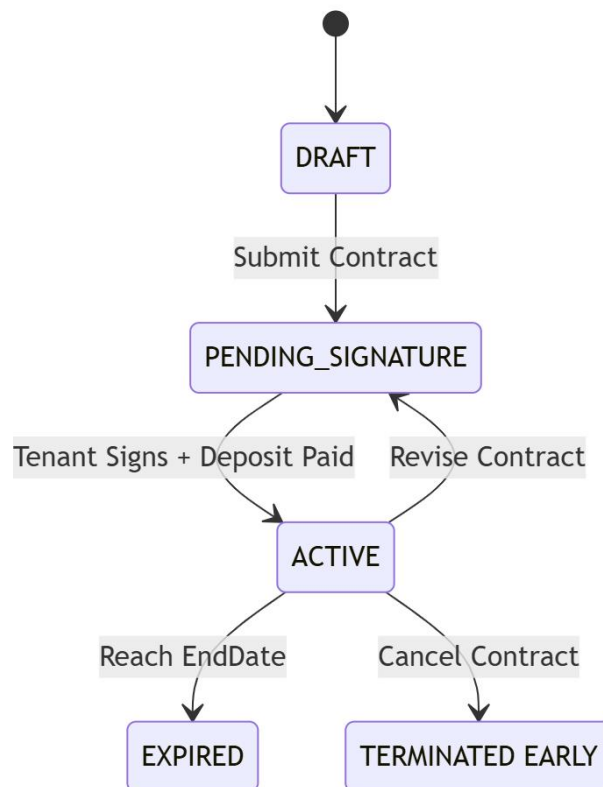
V.3.3. Contract Creation Algorithm

```
FUNCTION createContract(req):  
  1. Validate roomID and tenantID are positive integers  
  2. Validate startDate and endDate are valid ISO 8601 date  
strings  
  3. Assert endDate > startDate (else HTTP 400  
INVALID_PAYLOAD)  
  4. Validate deposit ≥ 0 and rentalPrice > 0 (else HTTP 400)  
  5. Fetch tenant: SELECT FROM users WHERE id=tenantID  
    → Assert tenant exists AND role IN ('tenant','Tenant')  
    → Else HTTP 404 TENANT_INVALID  
  6. Fetch room: SELECT FROM rooms WHERE id=roomID  
    → Assert room exists AND room.landlord_id = req.user.id  
    → Else HTTP 404 ROOM_NOT_FOUND  
    → Assert room.status = 'available'  
    → Else HTTP 400 ROOM_UNAVAILABLE  
  7. Check for existing active contract:  
    SELECT FROM lease_contracts WHERE tenant_id=tenantID AND  
status='active'  
    → If found: HTTP 400 TENANT_HAS_ACTIVE_CONTRACT  
  8. BEGIN TRANSACTION  
    a. INSERT INTO lease_contracts (tenant_id, room_id,  
start_date, end_date, deposit, rental_price,  
electricity_price, water_price, status='active',  
is_expired=0)  
    b. UPDATE rooms SET status='occupied' WHERE id=roomID  
  9. COMMIT  
  10. Return HTTP 201 with new contractID
```

V.3.4. Contract Termination Algorithm

```
FUNCTION terminateContract(contractId, req):  
  1. SELECT lc.id, lc.room_id, lc.status, r.landlord_id  
     FROM lease_contracts lc JOIN rooms r  
     WHERE lc.id=contractId AND r.landlord_id=req.user.id  
     → Not found: HTTP 404 CONTRACT_NOT_FOUND  
     → status ≠ 'active': HTTP 400 CONTRACT_NOT_ACTIVE  
  2. BEGIN TRANSACTION  
     a. UPDATE lease_contracts SET status='terminated' WHERE  
        id=contractId  
     b. UPDATE rooms SET status='available' WHERE id=room_id  
  3. COMMIT  
  4. Return HTTP 200 success message
```

V.3.5. Contract Lifecycle State Machine



V.4. Billing & Invoicing Component (C-04)

V.4.1. Purpose

This is the computational core of RPMS. It handles meter reading ingestion, invoice amount calculation via the invoiceCalculator service, invoice creation and persistence, payment recording, and PDF export.

V.4.2. Interfaces

Meter Reading Endpoints:

Method	Endpoint	Auth	Role	Description
POST	/api/meter-readings	Yes	landlord	Record new meter reading for a room
GET	/api/landlord/rooms/:roomID/previous-reading	Yes	landlord	Retrieve previous period reading for invoice pre-fill

Invoice Endpoints:

Method	Endpoint	Auth	Role	Description
POST	/api/landlord/invoices/calculate	Yes	landlord	Preview invoice calculation (no DB write)
GET	/api/invoices	Yes	landlord	List invoices (filter by room, month, year, status)
POST	/api/invoices	Yes	landlord	Create and persist a new invoice
GET	/api/invoices/:id	Yes	landlord/tenant	Retrieve invoice detail
PUT	/api/invoices/:id/pay	Yes	landlord	Mark invoice as fully paid

PATCH	/api/invoices/:id	Yes	landlord	Partial update of invoice status/payment fields
DELETE	/api/invoices/:id	Yes	landlord	Delete unpaid invoice and reset meter readings
GET	/api/invoices/mock-payment/:id	No	—	Public endpoint: invoice data for QR payment page
POST	/api/invoices/mock-payment/confirm	No	—	Public endpoint: confirm QR payment

V.4.3. Invoice Calculation Algorithm (invoiceCalculator.js)

This is the core billing computation, implemented in services/invoiceCalculator.js as a pure function calculateInvoiceTotal(input).

Input Schema:

```
InvoiceCalculationInput {
  roomPrice: Float (≥ 0),
  prevElectricityIndex: Float (≥ 0),
  currentElectricityIndex: Float (≥ prevElectricityIndex),
  prevWaterIndex: Float (≥ 0),
  currentWaterIndex: Float (≥ prevWaterIndex),
  serviceFees: {
    wifiFee: Float (≥ 0),
    trashFee: Float (≥ 0)
  },
  serviceUnitPrices: {
    electricityUnitPrice: Float (≥ 0, required),
    waterUnitPrice: Float (≥ 0, required)
  }
}
```

Computation Steps:

```
FUNCTION calculateInvoiceTotal(input):
```

```
    STEP 1 – Input Validation:
```

```
        FOR each numeric field IN [roomPrice, prevElectricityIndex,
currentElectricityIndex, prevWaterIndex, currentWaterIndex,
wifiFee, trashFee]:
```

```
            IF NOT isFinite(field) OR field < 0:
```

```
                THROW BillingValidationError(field + " must be a valid
non-negative number")
```

```
    STEP 2 – Service Price Validation:
```

```
        IF NOT isFinite(electricityUnitPrice) OR
electricityUnitPrice < 0:
```

```
            ADD 'electricityUnitPrice' to missingServicePricing
```

```
        IF NOT isFinite(waterUnitPrice) OR waterUnitPrice < 0:
```

```
            ADD 'waterUnitPrice' to missingServicePricing
```

```
        IF missingServicePricing.length > 0:
```

```
            THROW ServicePricingConfigError(missingFields)
```

```
    STEP 3 – Consumption Validation:
```

```
        IF currentElectricityIndex < prevElectricityIndex:
```

```
            THROW BillingValidationError("Electricity index cannot
decrease")
```

```
        IF currentWaterIndex < prevWaterIndex:
```

```
            THROW BillingValidationError("Water index cannot
decrease")
```

```
    STEP 4 – Consumption Calculation:
```

```
        electricityUsage = currentElectricityIndex -
prevElectricityIndex
```

```
        waterUsage      = currentWaterIndex - prevWaterIndex
```

```
    STEP 5 – Charge Calculation:
```

```
        electricityAmount = electricityUsage × electricityUnitPrice
```

```
        waterAmount      = waterUsage × waterUnitPrice
```

```
        serviceAmount    = wifiFee + trashFee
```

STEP 6 – Total Calculation:

```
totalAmount = roomPrice + electricityAmount + waterAmount + serviceAmount
```

```
RETURN {  
  breakdown: {  
    roomPrice, electricityUsage, electricityUnitPrice,  
    electricityAmount, waterUsage, waterUnitPrice, waterAmount,  
    wifiFee, trashFee, serviceAmount  
  },  
  totalAmount  
}
```

V.4.4. Previous Meter Reading Retrieval Algorithm (meterReadingService.js)

```
FUNCTION getPreviousMeterReading(roomID, month, year):  
  targetDate = YYYY-MM-01 formatted from (year, month)
```

QUERY:

```
  SELECT mr.id, mr.room_id, mr.electricity_index,  
  mr.water_index,  
         mr.recorded_date, i.month, i.year  
  FROM meter_readings mr  
  LEFT JOIN invoices i ON i.id = mr.invoice_id  
  WHERE mr.room_id = roomID  
        AND date(mr.recorded_date) < date(targetDate)  
  ORDER BY date(mr.recorded_date) DESC, mr.id DESC  
  LIMIT 1
```

```
  RETURN previousReading OR null (if no prior reading exists)
```

This algorithm ensures that the **most recent reading before the current billing period** is always used as the starting index, regardless of whether an invoice was generated in the immediately preceding month (handling gaps in billing periods correctly).

V.4.5. Invoice Creation Workflow

```
FUNCTION createInvoice(req):
  1. Validate roomID is a positive integer (else HTTP 400)
  2. Validate totalAmount is finite and  $\geq 0$  (else HTTP 400)
  3. Fetch room: SELECT FROM rooms WHERE id=roomID AND
landlord_id=req.user.id
    → Not found: HTTP 404 ROOM_NOT_FOUND
  4. IF contractID provided:
    Fetch contract: SELECT lc.id, lc.electricity_price,
lc.water_price, lc.start_date FROM lease_contracts lc
    JOIN rooms r WHERE lc.id=contractID AND
r.landlord_id=req.user.id
    AND lc.status='active'
    IF contract found:
      a. VALIDATE billing period  $\geq$  contract start date
        invoiceDate = new Date(year, month-1, 1)
        contractStart = new Date(contract.start_date)
        IF invoiceDate < contractStart:
          RETURN HTTP 400 BILLING_PERIOD_INVALID
      b. Sync electricity/water prices from contract if  $> 0$ 
  5. IF invoice contains electricity or water amounts  $> 0$ :
    CHECK uniqueness: SELECT id FROM invoices
    WHERE room_id=roomID AND
month=invoiceMonth AND year=invoiceYear
    AND (electricity_amount  $> 0$  OR
water_amount  $> 0$ )
    → If found: HTTP 400 DUPLICATE_UTILITY_BILL
  6. BEGIN TRANSACTION
    a. INSERT INTO invoices (room_id, contract_id,
reading_id, month, year, rent_amount, electricity_amount,
water_amount, service_amount, total_amount, due_date,
status='unpaid', payment_status='Unpaid')
    b. IF readingID provided:
      UPDATE meter_readings SET invoice_id = newInvoiceId
WHERE id=readingID
  7. COMMIT
  8. Return HTTP 201 with invoice ID
```


V.4.6. Payment Recording Algorithm

Full Payment (PUT /api/invoices/:id/pay):

```
FUNCTION markAsPaid(invoiceId, payment_method, req):  
  1. Validate invoiceId is a positive integer  
  2. Validate payment_method is non-empty string  
  3. SELECT i.id, i.status, r.landlord_id FROM invoices i  
  JOIN rooms r  
    WHERE i.id=invoiceId → Verify ownership  
  4. IF inv.status = 'paid': RETURN HTTP 400 ALREADY_PAID  
  5. UPDATE invoices SET  
    status='paid', payment_status='Paid',  
    payment_method=payment_method,  
    paid_amount=total_amount,  
    paid_at=CURRENT_TIMESTAMP  
    WHERE id=invoiceId  
  6. Return HTTP 200 success
```

Partial / QR Payment (POST /api/invoices/mock-payment/confirm):

```
FUNCTION confirmPartialPayment(invoiceId, amount):  
  1. Validate invoiceId > 0 and amount > 0  
  2. SELECT id, total_amount, paid_amount, status FROM  
  invoices WHERE id=invoiceId  
  3. IF invoice.status='paid' OR paid_amount >= total_amount:  
  RETURN 400 ALREADY_PAID  
  4. newPaidAmount = paid_amount + amount  
  5. isPaidInFull = (newPaidAmount >= total_amount)  
  6. nextStatus = isPaidInFull ? 'paid' : 'partial'  
  7. UPDATE invoices SET  
    status=nextStatus, payment_status=(isPaidInFull ?  
'Paid' : 'Partial'),  
    payment_method='Mock QR Transfer',  
    paid_amount=newPaidAmount,  
    paid_at=(isPaidInFull ? CURRENT_TIMESTAMP : paid_at)  
    WHERE id=invoiceId  
  8. Return { paidAmount, remainingAmount, invoiceStatus }
```

V.4.7. Invoice Deletion Algorithm

```
FUNCTION deleteInvoice(invoiceId, req):  
  1. SELECT i.id, i.status, i.reading_id, i.month, i.year,  
     i.room_id, r.landlord_id  
     FROM invoices i JOIN rooms r WHERE i.id=invoiceId AND  
     r.landlord_id=req.user.id  
     → Not found: HTTP 404  
  2. IF status = 'paid': RETURN HTTP 400  
     CANNOT_DELETE_PAID_INVOICE  
  3. BEGIN TRANSACTION  
     a. monthStart = YYYY-MM-01  
        monthEnd = last day of the billing month  
        UPDATE meter_readings SET electricity_index=0,  
water_index=0, invoice_id=NULL  
        WHERE room_id=inv.room_id AND recorded_date BETWEEN  
monthStart AND monthEnd  
     b. DELETE FROM invoices WHERE id=invoiceId  
  4. COMMIT  
  5. Return HTTP 200
```

V.5. Tenant Portal Component (C-05)

V.5.1. Purpose

Provides all tenant-facing API endpoints enabling authenticated Tenants to view their contract, invoices, payment history, submit maintenance requests, and manage their profile. All endpoints enforce tenant-scoped data isolation.

V.5.2. Interfaces

Method	Endpoint	Auth	Role	Description
GET	/api/tenants/contract	Yes	tenant	Own active contract
GET	/api/contracts/my- contract	Yes	any	Active contract (via contracts router)
GET	/api/tenants/invoices	Yes	tenant	All invoices linked to own contract

GET	/api/tenants/invoices/:id	Yes	tenant	Invoice detail with meter reading data
POST	/api/tenants/invoices/:id/pay	Yes	tenant	Pay an invoice
GET	/api/tenants/maintenance	Yes	tenant	Own maintenance requests
POST	/api/tenants/maintenance	Yes	tenant	Submit new maintenance request
GET	/api/maintenance-requests	Yes	tenant/landlord	List maintenance requests (role-filtered)
POST	/api/maintenance-requests	Yes	any	Submit maintenance request (with photo)

V.5.3. Tenant Invoice Query Algorithm

The tenant invoice endpoint enforces data isolation by joining through `lease_contracts`:

```
SELECT i.*, r.name AS room_name
FROM invoices i
JOIN lease_contracts lc ON i.contract_id = lc.id
JOIN rooms r ON lc.room_id = r.id
WHERE lc.tenant_id = req.user.id
ORDER BY i.created_at DESC
```

Invoice detail additionally joins `meter_readings` for the most recent reading linked to that invoice:

```

SELECT i.*, r.name, r.price,
       mr.electricity_index AS curr_elec, mr.water_index AS
curr_water,
       mr.prev_electricity_index AS prev_elec,
mr.prev_water_index AS prev_water
FROM invoices i
JOIN lease_contracts lc ON i.contract_id = lc.id
JOIN rooms r ON lc.room_id = r.id
LEFT JOIN meter_readings mr ON mr.id = (
    SELECT mr2.id FROM meter_readings mr2
    WHERE mr2.invoice_id = i.id ORDER BY mr2.id DESC LIMIT 1
)
WHERE i.id = invoiceId AND lc.tenant_id = req.user.id

```

V.5.4. Maintenance Submission Algorithm

```

FUNCTION submitMaintenanceRequest(req):
    1. Extract description, category from req.body
    2. VALIDATE description.trim().length > 0 (else HTTP 400)
    3. Handle optional photo:
        IF req.file EXISTS (multipart upload):
            Convert to Base64: base64Image =
file.buffer.toString('base64')
            issuePhoto = "data:" + file.mimetype + ";base64," +
base64Image
        ELSE issuePhoto = null
    4. Retrieve active contract:
        SELECT id, room_id FROM lease_contracts
        WHERE tenant_id = req.user.id AND status='active' LIMIT
1
        → Not found: HTTP 400 "No active lease contract"
    5. Auto-priority assignment:
        highPriorityKeywords = ['gas','điện
giật','cháy','nổ','nguy hiểm','khẩn cấp','rò rỉ']
        IF description.toLowerCase() contains ANY keyword:
            priority = 'high'
        ELSE:
            priority = 'normal'

```

```
6. INSERT INTO maintenance_requests
   (contract_id, room_id, tenant_id, description, category,
    priority, status='pending', issue_photo)
7. Return HTTP 201 with requestId and priority
```

V.6. Reporting & Analytics Component (C-06)

V.6.1. Purpose

Generates financial dashboards, computes revenue and bad-debt statistics, and produces government-mandated legal documents in PDF and XLSX formats.

V.6.2. Interfaces

Method	Endpoint	Auth	Role	Description
GET	/api/landlord/financial-stats	Yes	landlord	Revenue, bad debt, and occupancy summary
GET	/api/reports/temporary-residence	No	—	Export Temporary Residence Report (PDF)
GET	/api/reports/tax	No	—	Export Tax Declaration Report (XLSX)

V.6.3. Financial Statistics Algorithm

```
FUNCTION getFinancialStats(landlordId):
```

```
    Query 1 – Total Revenue (paid invoices):
```

```
        SELECT SUM(paid_amount) AS totalRevenue
        FROM invoices i JOIN rooms r ON i.room_id = r.id
        WHERE r.landlord_id = landlordId AND i.status = 'paid'
```

```
    Query 2 – Total Bad Debt (unpaid balances):
```

```
        SELECT SUM(total_amount - paid_amount) AS totalBadDebt
        FROM invoices i JOIN rooms r ON i.room_id = r.id
        WHERE r.landlord_id = landlordId AND i.status IN
        ('unpaid', 'partial')
```

```
    Query 3 – Room Occupancy:
```

```
        SELECT COUNT(*) AS totalRooms,
               SUM(CASE WHEN status='occupied' THEN 1 ELSE 0
END) AS occupiedRooms
        FROM rooms WHERE landlord_id = landlordId
```

```
    COMPUTE:
```

```
        occupancyRate = (occupiedRooms / totalRooms) × 100,
rounded to nearest integer
        OR 0 if totalRooms = 0
```

```
    RETURN {
```

```
        totalRevenue,
        badDebt: totalBadDebt,
        occupancyRate,
        totalRooms
```

```
    }
```

V.6.4. Temporary Residence Report Generation Algorithm

```
FUNCTION generateTemporaryResidenceReport():  
  1. QUERY active tenants:  
    SELECT u.full_name, u.citizen_id, u.date_of_birth,  
           u.permanent_address, c.start_date  
    FROM users u  
    JOIN lease_contracts c ON u.id = c.tenant_id  
    WHERE c.status = 'active' OR c.is_expired = 0  
    GROUP BY u.id  
  
  2. Initialise PDFDocument (A4, margin: 40pt)  
    Register fonts: arial.ttf (regular), arialbd.ttf (bold)  
  
  3. RENDER HEADER SECTION:  
    Line 1 (bold, 13pt, centred): "CỘNG HÒA XÃ HỘI CHỦ NGHĨA  
VIỆT NAM"  
    Line 2 (bold, 13pt, centred): "Độc lập - Tự do - Hạnh phúc"  
    Line 3 (bold, 14pt, centred): "DANH SÁCH NGƯỜI LƯU TRÚ KÈM  
THEO"  
    Line 4 (italic, 12pt, centred): "(Phụ lục cho thông báo lưu  
trú)"  
  
  4. RENDER TABLE via pdfkit-table:  
    Columns: STT (30pt), Họ và tên (120pt), Ngày sinh (70pt),  
Số CCCD (90pt), Quê quán (130pt), Ngày chuyển vào (75pt)  
    Row data: tenants mapped from query result  
    Header style: bold, 10pt; Row style: regular, 10pt  
  
  5. RENDER FOOTER:  
    Right-aligned: "Người khai báo lưu trú" and "(Ký, ghi rõ họ  
tên)"  
  
  6. SET response headers:  
    Content-Type: application/pdf  
    Content-Disposition: attachment;  
filename=Danh_sach_tam_tru.pdf  
  
  7. Pipe doc to res (stream to client)
```

V.6.5. Tax Report Generation Algorithm

```
FUNCTION generateTaxReport():  
  1. QUERY paid invoices:  
    SELECT i.month, i.year, r.name AS room_name,  
           i.rent_amount, i.electricity_amount,  
i.water_amount,  
           i.service_amount, i.total_amount  
    FROM invoices i JOIN rooms r ON i.room_id = r.id  
    WHERE LOWER(i.status) = 'paid' OR LOWER(i.payment_status)  
= 'paid'  
    ORDER BY i.year DESC, i.month DESC, r.name ASC  
  
  2. Create ExcelJS Workbook and Worksheet "Báo Cáo Thuế"  
  3. Define columns:  
    month_year (15), room (20), rent (15), electric (15),  
    water (15), service (15), total (20)  
  4. Style header row: bold, centred  
  5. FOR each invoice: addRow with formatted data  
  6. ADD empty row, then grand total row (bold)  
  7. Apply Vietnamese number format (#,##0) to all monetary  
columns  
  
  8. SET response headers:  
    Content-Type: application/vnd.openxmlformats-  
officedocument.spreadsheetml.sheet  
    Content-Disposition: attachment;  
filename=Bao_cao_thue.xlsx  
  
  9. await workbook.xlsx.write(res)
```

V.7. Authentication Service Component (C-07)

V.7.1. Purpose

Implements the complete authentication subsystem including JWT issuance, validation, Refresh Token Rotation, logout, token revocation, password management, and rate limiting.

V.7.2. Token Architecture

The system uses a **dual-token strategy**:

Token	Lifetime	Storage	Purpose
Access Token (JWT)	15 minutes	Client memory / HTTP header	Authorise API calls
Refresh Token (JWT)	7 days	Client storage + DB (refresh_tokens)	Issue new Access Tokens

Access Token Payload Structure:

```
{
  "id": 42,
  "email": "landlord@test.com",
  "role": "landlord",
  "name": "Test Landlord",
  "jti": "uuid-v4-unique-string",
  "type": "access",
  "iat": 1716523200,
  "exp": 1716524100
}
```

Refresh Token Payload Structure:

```
{
  "id": 42,
  "jti": "uuid-v4-unique-string",
  "type": "refresh",
  "iat": 1716523200,
  "exp": 1717128000
}
```

V.7.3. Token Validation Middleware (middleware/auth.js)

FUNCTION authenticateToken(req, res, next):

1. Extract token from Authorization header: "Bearer <token>"
→ No token: HTTP 401 "Access token required"
2. jwt.verify(token, JWT_SECRET)
→ Verification error mentioning 'required': HTTP 500 (config error)
→ Other error: HTTP 403 "Invalid or expired token"
3. ASSERT decoded.type = 'access'
→ Wrong type: HTTP 403 "Invalid token type"
4. ASSERT decoded.jti is present
→ Missing: HTTP 403 "Invalid token payload"
5. SELECT jti FROM revoked_access_tokens WHERE jti = decoded.jti
→ Found in revocation list: HTTP 403 "Invalid or expired token"
6. req.user = decoded
7. CALL next()

V.7.4. Login Algorithm

FUNCTION login(email, password):

1. Normalise email: trim + lowercase
2. Validate email format (EMAIL_REGEX) → HTTP 400 on failure
3. SELECT user WHERE email = normalizedEmail
→ Not found: HTTP 401 "Email hoặc mật khẩu không đúng"
4. IF user.status = 'pending': HTTP 403 "Chờ Chủ trọ phê duyệt"
IF user.status = 'inactive': HTTP 403 "Tài khoản đã bị khóa"
5. bcrypt.compare(password, user.password)
→ Mismatch: HTTP 401 "Email hoặc mật khẩu không đúng"
6. CALL issueTokens(user):
 - a. Generate accessJti = crypto.randomUUID()
Generate refreshJti = crypto.randomUUID()
 - b. Sign Access Token (HS256, 15m expiry, includes jti)
 - c. Sign Refresh Token (HS256, 7d expiry, includes jti)
 - d. Hash Refresh Token: SHA-256(rawRefreshToken)
 - e. INSERT INTO refresh_tokens (user_id, token_hash, jti, expires_at)
7. Return { token, refreshToken, user: toPublicUser(user) }

V.7.5. Refresh Token Rotation Algorithm

```
FUNCTION refreshAccessToken(rawRefreshToken):  
  1. jwt.verify(rawRefreshToken, JWT_REFRESH_SECRET)  
    → Error: HTTP 401 "Refresh token không hợp lệ hoặc đã  
    hết hạn"  
  2. ASSERT decoded.type = 'refresh'  
  3. currentTokenHash = SHA-256(rawRefreshToken)  
  4. SELECT * FROM refresh_tokens  
    WHERE token_hash=currentTokenHash AND user_id=decoded.id  
  AND revoked=0  
    → Not found: HTTP 401 "Refresh token đã bị thu hồi"  
  5. IF tokenRow.expires_at <= NOW():  
    UPDATE refresh_tokens SET revoked=1, revoked_at=NOW()  
  WHERE id=tokenRow.id  
    RETURN HTTP 401 "Refresh token đã hết hạn"  
  6. SELECT user WHERE id=decoded.id  
    → Verify user exists and status is active  
  7. Generate newAccessJti = crypto.randomUUID()  
    Generate newRefreshJti = crypto.randomUUID()  
  8. Sign new Access Token and new Refresh Token  
  9. UPDATE refresh_tokens SET  
    revoked=1, revoked_at=NOW(),  
    replaced_by_jti=newRefreshJti,  
    last_used_at=NOW()  
    WHERE id=tokenRow.id  
  10. INSERT new Refresh Token record into refresh_tokens  
  11. Return { token: newAccessToken, refreshToken:  
  newRefreshToken, user }
```

V.7.6. Logout Algorithm

```
FUNCTION logout(req, rawRefreshToken):  
  1. IF req.user.jti AND req.user.exp:  
    accessExpiresAt = new Date(req.user.exp * 1000)  
    INSERT OR IGNORE INTO revoked_access_tokens  
      (jti, user_id, expires_at) VALUES (req.user.jti,  
req.user.id, accessExpiresAt)  
  2. IF rawRefreshToken provided:  
    tokenHash = SHA-256(rawRefreshToken)  
    UPDATE refresh_tokens SET revoked=1, revoked_at=NOW()  
    WHERE token_hash=tokenHash AND user_id=req.user.id AND  
revoked=0  
  3. Return HTTP 200 "Đăng xuất thành công"
```

V.7.7. Token Cleanup Background Job

Runs every 3,600,000 ms (60 minutes):

```
JOB tokenCleanup:  
  DELETE FROM revoked_access_tokens WHERE expires_at <  
datetime('now')  
  DELETE FROM refresh_tokens  
  WHERE revoked=1 AND revoked_at < datetime('now', '-7 days')
```

V.8. Notification Service Component (C-08)

V.8.1. Purpose

Manages all system-generated and manually authored notifications, supporting broadcast to all tenants, room-level targeting, individual targeting, and custom selection targeting.

V.8.2. Interfaces

Method	Endpoint	Auth	Role	Description
POST	/api/notifications	Yes	landlord	Send notification (broadcast or targeted)

GET	/api/notifications	Yes	landlord/tenant	List notifications (role-filtered)
GET	/api/notifications/unread-count	Yes	any	Count unread notifications
PUT	/api/notifications/:id/read	Yes	any	Mark notification as read
DELETE	/api/notifications/:id	Yes	any	Delete a notification

V.8.3. Notification Dispatch Algorithm

```
FUNCTION sendNotification(title, message, recipientType,
recipientId, selectedTenantIds, req):
```

```
    VALIDATE:
```

- title.trim().length > 0 and title.length ≤ 200
- message.trim().length > 0 and message.length ≤ 2000
- recipientType IN

```
('all_tenants', 'room', 'tenant', 'selected_tenants')
```

```
    BEGIN TRANSACTION
```

```
    CASE recipientType:
```

```
        'all_tenants':
```

```
            SELECT DISTINCT lc.tenant_id FROM lease_contracts lc
            JOIN rooms r WHERE r.landlord_id=req.user.id AND
lc.status='active'
```

```
            FOR each tenant:
```

```
                INSERT INTO notifications (from_user_id,
recipient_type, recipient_id, title, message)
                RETURN { notificationCount: tenants.length }
```

```
        'room':
```

```
            VALIDATE recipientId (roomId) > 0
```

```
            VERIFY room.landlord_id = req.user.id
```

```
            SELECT DISTINCT lc.tenant_id FROM lease_contracts
            WHERE room_id=recipientId AND status='active'
```

```
            FOR each tenant: INSERT notification
```

```

'tenant':
    VALIDATE recipientId (tenantId) > 0
    VERIFY tenant has active lease in landlord's property
    INSERT single notification WHERE recipient_id =
recipientId
    RETURN { notificationId }

'selected_tenants':
    VALIDATE selectedTenantIds is non-empty array
    SELECT valid tenants: intersect selectedTenantIds with
active-contract tenants
    FOR each valid tenant: INSERT notification
    RETURN { notificationCount: validTenants.length }

COMMIT

```

V.8.4. Notification Retrieval Logic

The retrieval algorithm applies role-based filtering:

```

IF userRole = 'landlord':
    SELECT * FROM notifications WHERE from_user_id = userId
    ORDER BY created_at DESC

IF userRole = 'tenant':
    SELECT * FROM notifications
    WHERE recipient_type = 'all_tenants' OR recipient_id = userId
    ORDER BY created_at DESC

```

Optional read-status filter: AND is_read = (1 if isRead='true', 0 if isRead='false')

V.9. Audit & Logging Component (C-09)

V.9.1. Purpose

Provides immutable audit trail recording of all significant state-mutating operations in the system, enabling compliance verification and forensic investigation.

V.9.2. Interfaces

Method	Endpoint	Auth	Role	Description
GET	/api/audit-logs	Yes	landlord	Query audit logs with filters and pagination
GET	/api/audit-logs/filters/actions	Yes	landlord	List distinct action types
GET	/api/audit-logs/filters/entity-types	Yes	landlord	List distinct entity types

V.9.3. Middleware Audit Capture (middleware/audit.js)

The `auditMutations(targetTable)` middleware is applied as a pre-processing layer on PUT and DELETE routes for rooms and lease_contracts.

```
FUNCTION auditMutations(targetTable):  
  VALIDATE targetTable IN ('rooms', 'lease_contracts')  
  
  RETURN middleware(req, res, next):  
    method = req.method.toUpperCase()  
    IF method NOT IN ('PUT', 'DELETE'): CALL next(); RETURN  
  
    auditEntry = {  
      userId: req.user.id (or null),  
      action: method = 'PUT' ? 'UPDATE' : 'DELETE',  
      targetTable: targetTable,  
      targetId: req.params.id OR req.body.id OR null,  
      path: req.originalUrl,  
      method: method,  
      payload: { body, params, query } (non-empty fields only),  
      createdAt: new Date().toISOString()  
    }  
  
    res.on('finish', () => {  
      auditEntry.statusCode = res.statusCode  
      db.insertAuditLogAsync(auditEntry)  
        .catch(err => console.error('Audit log write failure:',  
err))  
    })
```

Key design: the `res.on('finish')` hook captures the response status code **after** the route handler completes, ensuring the audit log records whether the mutation succeeded or failed.

V.9.4. Audit Log Query Algorithm

```
FUNCTION getAuditLogs(landlordId, filters):
  { action, entityType, startDate, endDate, limit=100, offset=0
} = filters
  CLAMP limit to maximum 500
  FLOOR offset to minimum 0

  BASE QUERY:
    SELECT al.id, al.user_id, al.landlord_id, al.action,
    al.entity_type,
           al.entity_id, al.description, al.old_values,
    al.new_values,
           al.status, al.created_at, u.name AS user_name,
    u.email AS user_email
    FROM audit_logs al LEFT JOIN users u ON al.user_id = u.id
    WHERE al.landlord_id = landlordId

  APPLY optional filters:
    IF action:      AND al.action = action
    IF entityType: AND al.entity_type = entityType
    IF startDate:   AND DATE(al.created_at) >= DATE(startDate)
    IF endDate:    AND DATE(al.created_at) <= DATE(endDate)

  ORDER BY al.created_at DESC
  LIMIT limit OFFSET offset

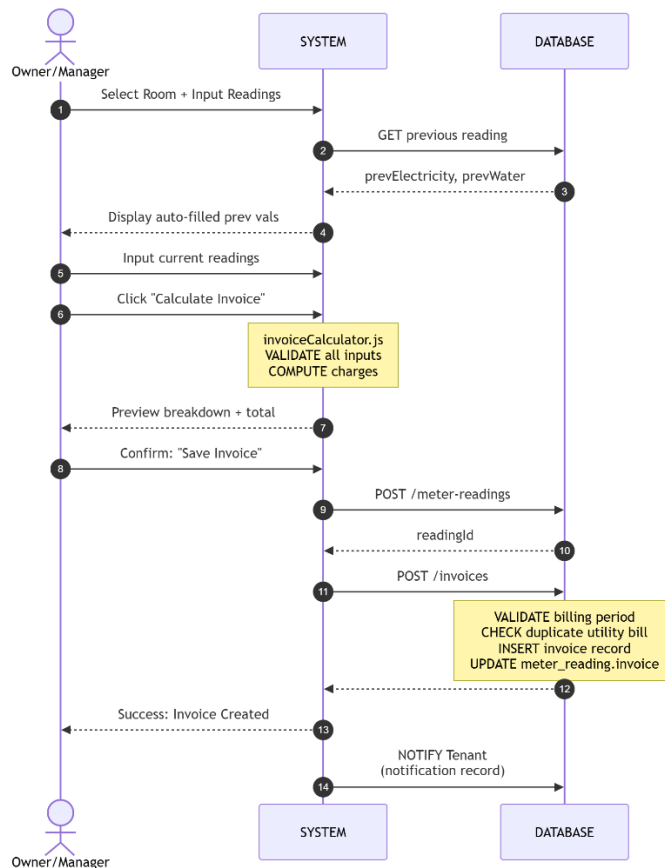
  POST-PROCESS:
    FOR each log: Parse old_values and new_values from JSON
    strings

  RETURN logs
```

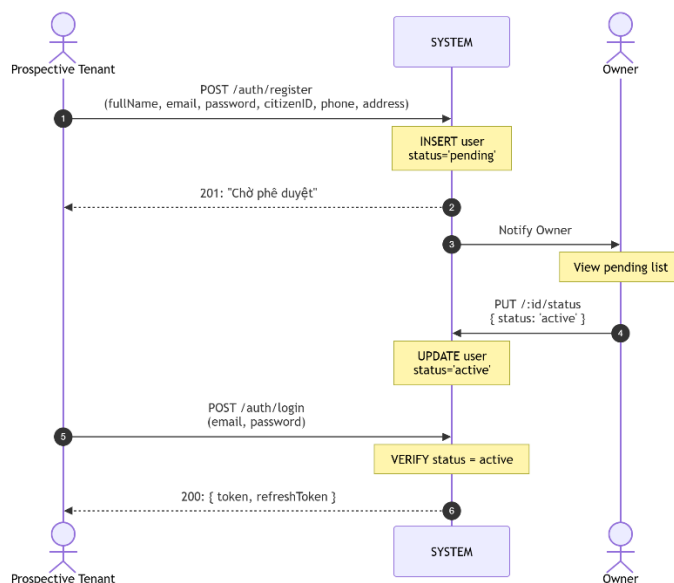

VI. Inter-Component Workflow Descriptions

VI.1. Monthly Billing Workflow (UC-02)

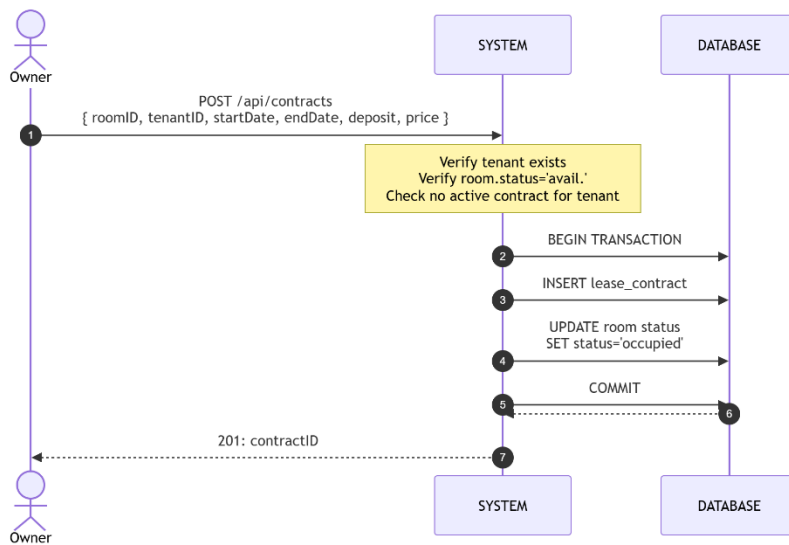
This workflow describes the complete end-to-end process from meter reading to invoice payment confirmation.



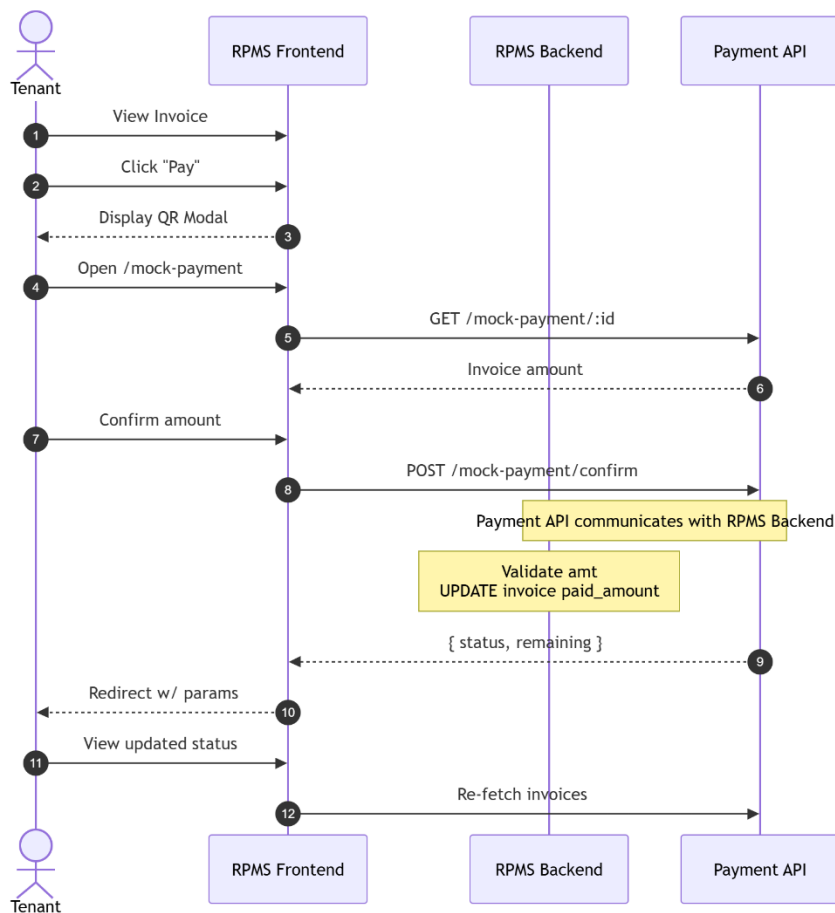
VI.2. Tenant Registration & Approval Workflow (UC-09, UC-01)



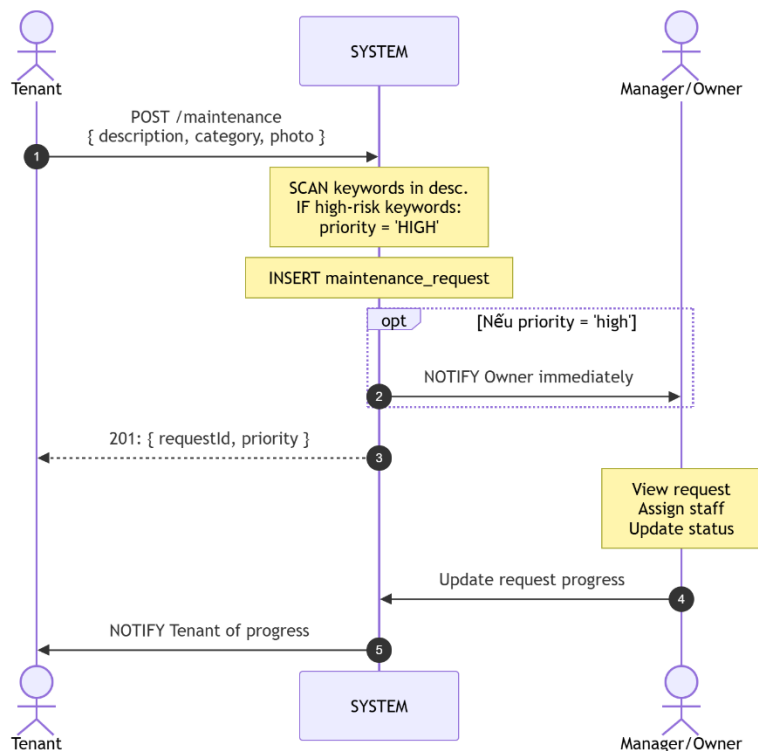
VI.3. Contract Creation & Room Status Transition Workflow (UC-08)



VI.4. QR Code Payment Workflow (UC-03)



VI.5. Maintenance Request Priority Escalation Workflow (UC-04)



VII. COTS/SOUP Component Identification

The following third-party libraries are used in the RPMS backend and frontend. All versions are locked in package.json.

VII.1. Backend Dependencies

Library	Version	Purpose	Risk Category
express	4.x	HTTP server framework, routing, middleware	Low
sqlite3	5.x	SQLite3 database driver with verbose mode	Low
jsonwebtoken	—	JWT signing and verification (HS256 algorithm)	Medium — cryptographic
bcryptjs	2.x	Password hashing with bcrypt algorithm (cost factor 10)	Medium — cryptographic

express-rate-limit	—	Request rate limiting middleware	Low
cors	—	Cross-Origin Resource Sharing header management	Low
multer	—	Multipart form-data file upload handling	Low
pdfkit-table	—	PDF document generation with table support, Vietnamese font rendering	Low
exceljs	4.x	XLSX spreadsheet generation for tax reports	Low
dotenv	—	Environment variable loading from .env files	Low
crypto	Node.js built-in	UUID generation (randomUUID), SHA-256 hashing	Low

VII.2. Backend Dependencies

Library	Version	Purpose	Risk Category
react	18.x	UI component framework	Low
react-dom	18.x	DOM rendering for React	Low
react-router-dom	—	Client-side routing, navigation	Low
axios	—	HTTP client with interceptors for JWT attachment and refresh	Low
recharts	2.x	Data visualisation: LineChart, BarChart, PieChart	Low
vite	—	Build tooling and development server	Low

VII.3. Vietnamese Font Files (Static Assets)

File	Type	Purpose
backend/fonts/arial.ttf	TrueType Font	Regular weight Vietnamese text in PDF reports
backend/fonts/arialbd.ttf	TrueType Font	Bold weight Vietnamese text in PDF reports

These font files are required for correct rendering of Vietnamese diacritical characters in the Temporary Residence Report PDF. Absence of these files will cause the report generation endpoint to fail.

VIII. Critical Requirements Traceability

The following table maps the highest-priority system requirements to their implementing components, algorithms, and verifiable acceptance criteria.

Req ID	Requirement Summary	Implementing Component	Algorithm/ Section	Acceptance Criterion
FR-INV-001	Auto-calculate invoice total using room + electricity + water + service fees	C-04	§5.4.3 calculateInvoiceTotal()	Unit test: given known indices and prices, output matches expected totalAmount
FR-INV-004	Reject invoices with billing period before contract start date	C-04	§5.4.5 Step 4a	Test: attempt invoice for month/year before contract startDate returns HTTP 400 BILLING_PERIOD_INVALID

FR-INV-007	Prevent deletion of paid invoices	C-04	§5.4.7 Step 2	Test: DELETE on paid invoice returns HTTP 400 CANNOT_DELETE_PAID_INVOICE
FR-MTR-001/002/003	Reject meter readings lower than previous readings	C-04	§5.4 POST /meter-readings	Test: submit currentElectricity < prevElectricity returns HTTP 400 INVALID_INDEX_VALUES
FR-MTR-004	Auto-fill previous reading as current period start index	C-04	§5.4.4 getPreviousMeterReading()	Integration test: previous reading is correctly identified from DB across non-consecutive months
FR-CON-001/002/003	Contract creation atomically sets room to occupied	C-03	§5.3.3 Transaction Step 8	Test: after POST /contracts, room.status = 'occupied' in DB
FR-CON-003	Reject second active contract for same tenant	C-03	§5.3.3 Step 7	Test: second contract for same tenant returns HTTP 400 TENANT_HAS_ACTIVE_CONTRACT
FR-CFG-004	Refresh Token Rotation: old	C-07	§5.7.5 Algorithm	Test: re-use of consumed refresh

	token revoked, new token issued			token returns HTTP 401
FR-CFG-007	Role-based access control on all endpoints	C-07	middleware /auth.js requireRole()	Test: tenant role accessing /api/rooms returns HTTP 403
FR-MNT-003	Auto-assign high priority for hazard keywords	C-05	§5.5.4 Step 5	Test: description containing "cháy" results in priority='high' in DB
FR-PAY-004/005	Partial payment tracking; prevent over- payment	C-04	§5.4.6 confirmPartialPayment ()	Test: two partial payments summing to total_amount results in status='paid'
FR-RPT-001	Temporary Residence Report PDF with all active tenant data	C-06	§5.6.4 Algorithm	Demonstration: PDF contains correct tenant records, rendered in Vietnamese with proper formatting
FR-RPT-003	Tax report excludes unpaid invoices	C-06	§5.6.5 Step 1 SQL WHERE	Test: unpaid invoice rooms do not appear in exported XLSX
FR-AUD-001/002	Audit middleware captures PUT/DELETE	C-09	§5.9.3 Algorithm	Inspection: verify audit_logs table entry exists after PUT /rooms/:id

	on rooms and contracts			
FR-SEC-08	All payment data encrypted at rest	C-07	BR-SEC-08	Inspection: payment_method and transaction data fields use appropriate storage encryption
FR-AUD-006	System operates 24/7	Cross-cutting	server.js keepAliveTimeout	Analysis: keepAliveTimeout=65s, headersTimeout=66s prevent proxy-level disconnects
FR-AUD-005	Response time < 4.5 seconds	Cross-cutting	SQLite indexes on foreign keys	Performance test: common queries (invoice list, room list) complete in < 500ms under nominal load

IX. Design Constraints and Assumptions

Constraint ID	Description
DC-01	The system uses SQLite as the sole database engine. This constrains concurrent write throughput but is acceptable for single-property deployments. Migration to PostgreSQL would be required for multi-property enterprise deployments.

DC-02	The JWT_SECRET environment variable must be set before server startup. The application will not start without it (enforced by assertAuthConfig()).
DC-03	Vietnamese font files (arial.ttf, arialbd.ttf) must be present in backend/fonts/ for PDF generation. The server will not throw on startup but the /reports/temporary-residence endpoint will fail at runtime if these are absent.
DC-04	The frontend communicates exclusively with the backend via VITE_API_URL (default http://localhost:5000/api). The CORS policy on the backend must whitelist this origin via FRONTEND_URL.
DC-05	File uploads for maintenance request photos are limited to 5 MB and must be of MIME type image/jpeg, image/png, image/gif, or image/webp.
DC-06	The system is designed for Vietnamese language users. All error messages, UI text, and generated document headers are in Vietnamese.
DC-07	PRAGMA foreign_keys = ON is set at database initialisation, enforcing referential integrity at the SQLite engine level. All DELETE operations must comply with this constraint.